



# 第七章 文件

赵晓航

[xiaohangzhao@mail.shufe.edu.cn](mailto:xiaohangzhao@mail.shufe.edu.cn)

上海财经大学·信息管理与工程学院



# 目录

---

1. 文件的操作流程
2. 打开和关闭文件
3. 操作文本文件内容
4. 操作二进制文件内容
5. 序列化与反序列化
6. 文件的操作
7. 目录的操作
8. 应用案例
9. 本章习题



# 1. 文件的操作流程

- 文件是在磁盘上长期保存数据的基本存储方式
  - 即使在使用数据库时，数据库的数据最终也通常以文件的形式存储
  - 理解文件的操作对于任何涉及数据存储和处理的编程任务都至关重要
- 根据文件数据的组织形式，文件可以分为
  - 文本文件：存储的是可读的字符（如ASCII或Unicode字符），常见于文本编辑器中。文本文件由行组成，每行通常以特定的换行符（如\n）结束
  - 二进制文件：以字节形式存储数据，包括图像、音频、视频等文件类型。这些文件不是为了被人直接阅读而设计的，而是为了提供给计算机系统或特定软件处理



# 文件操作的基本流程

## • 文件操作的基本流程

1. **打开文件**：涉及到文件路径的指定以及打开模式的选择。打开模式决定了文件被访问的方式（如仅读取、写入、追加等）
2. **操作文件内容**：根据需要进行读取、写入、修改或删除操作。这些操作可以应用于文件的整个内容或仅其一部分
3. **关闭文件**：完成操作后关闭文件是一个好习惯。这不仅有助于释放系统资源，还可以确保所有写入操作得以保存

## • 文件类型及其操作

- **文本文件的操作**：使用内置的`open()`函数以文本模式打开文件。读取文本文件时，常用的方法包括`read()`, `readline()`, 和`readlines()`
- **二进制文件的操作**：打开二进制文件时，需要在`open()`函数中指定 `'b'` 模式。这种模式允许读取和写入字节数据，常用于图像、音频文件等



## 2. 打开和关闭文件

- 在Python中，文件操作主要通过内置的`open()`函数进行。该函数用于打开一个文件，并返回一个文件对象，后续可以通过该对象进行文件读写操作
- 打开文件：打开文件`open()`函数的基本语法是：
  - `open(file, mode='r' )`
    - `file`：指定要打开的文件的路径和名称。如果文件位于当前工作目录下，可以直接使用文件名。
    - `mode`：指定打开文件的模式

| 模式 | 说明                      |
|----|-------------------------|
| r  | 只读模式。如果文件不存在，抛出异常       |
| w  | 写入模式。如果文件存在，覆盖；不存在，创建   |
| a  | 追加模式。在文件末尾追加内容          |
| b  | 二进制模式。与其他模式组合使用（如rb或wb） |
| +  | 读写模式（可与其他模式组合使用）        |

- 函数返回值为文件对象，它是一个iterator对象



# 打开文件

- “+” 模式的使用：

- '+' 本身不是一个独立的文件模式，而是一个可以添加到其他模式（如 'r', 'w', 'a'）的修饰符，用以增加对文件的读写能力。这意味着，单独的 '+' 没有实际意义，必须与其他模式结合使用。例如，
- 'r+'：读写模式。文件必须存在，否则会抛出错误。可以读取文件，也可以写入新的内容，写入的内容会从文件开头开始覆盖原有内容，但不会截断文件（即不会删除从文件指针开始到文件末尾的内容）。
- 'w+'：读写模式。如果文件存在，则覆盖它；如果文件不存在，则创建新文件。这个模式可以读取也可以写入，与 'w' 的区别是增加了读取的能力。
- 'a+'：读写模式。如果文件存在，写入的数据将添加到文件末尾（不覆盖现有数据）；如果文件不存在，则创建新文件。这个模式也可以读取文件。



# 打开和关闭文件

- 例如，要以只读模式打开一个位于指定路径的文件：
- `f1 = open('d:/data/input/file1.txt', 'r' )`
- 而要在当前目录下创建或覆盖一个文件以进行写入：
- `f2 = open('file2.txt', 'w' )`
- 注意指定文件路径有三种常用方式：
  - **1. 使用文件名**：如果文件在当前目录下，可以直接使用文件名。
  - **2. 使用绝对路径**：提供文件的完整路径，适用于文件不在当前目录下的情况。
  - **3. 使用相对路径**：相对于当前脚本所在目录的文件路径。

## • 关闭文件

- 操作完成后，使用文件对象的**`close()`方法**关闭文件是一个良好的实践。这确保了所有操作都已完成，并且资源被正确释放
- `f1.close()`



# 上下文管理器

- 上下文管理器:

- 为了确保文件在使用后能够自动关闭, 建议使用with语句作为上下文管理器
- 这种方法在处理文件时更为安全, 因为它会在代码块执行完毕后自动关闭文件, 即使发生异常也是如此
- `with open('file2.txt', 'w') as f2:`
- `# 在这里进行文件操作`
- `pass`
- `# 文件在这里已自动关闭`





### 3. 操作文本文件内容

- **读文件：**读取文件的操作是将磁盘上的文件内容读入内存中
  - Python提供了多种方法来读取文件内容，以下是其中最常用的几种：

---

| 读取方法                              | 说明                                                                   |
|-----------------------------------|----------------------------------------------------------------------|
| <code>f.read(size=-1)</code>      | 读取前size个字符的内容，默认值为-1，表示读取整个文件                                        |
| <code>f.readline(size=-1)</code>  | 读取一行内容，size参数可以限制读取的字符数，默认为-1，表示读取整行                                 |
| <code>f.readlines(hint=-1)</code> | 读取文件中的行，并返回一个列表，hint 参数用于指定读取的字节数上限（按字节数估算，但始终返回完整的行），默认为 -1，表示读取所有行 |

---



## 举例：读文件

- 举例：以文本方式逐行读取文件内容
  - `file = open('./data/a.txt','r')`
  - `for line in file.readlines():`
  - `print(line)`
  - `file.close()`
  - 行2-3将文件全部内容从硬盘中读出，存储在内存的一个列表中，每一行内容作为列表的元素，读取每一行，并打印之
  - **上述代码中有一个性能缺陷**：当要读取的文件内容很大，将整个文件内容存储在一个列表中，会占据大量的内存，影响程序执行效率。



# 举例：读文件

- **举例：代码改进**

- 改进的方法是逐行读入内存，并逐行处理，减少内存消耗。Python将文件作为一个行序列，遍历文件所有行
- `file = open('./data/a.txt', 'r')`
- `for line in file:`
- `print(line)`
- `file.close()`
- 这段代码首先打开一个文本文件进行读取，然后逐行遍历文件中的每一行并打印出来。这种方法减少了内存的占用，但可能导致频繁的硬盘I/O操作，尤其是在处理非常大的文件时

- **进一步改进：**

- 频繁的I/O操作可能会影响程序的执行效率，因为每次读取操作都需要时间来从硬盘中读取数据。为了平衡内存使用和I/O效率，建议采用缓冲读取策略，例如，一次读取文件的一个较大块（例如几KB或几MB），然后再从这个大块中逐行处理数据。这样可以减少硬盘I/O操作的次数，同时保持内存使用在合理的范围内。



# 写文件

- **写文件：**

- 将内存中的数据，写入到磁盘的文件中。
- Python提供了几种写入文件内容的方法：

| 方法                               | 说明                                                    |
|----------------------------------|-------------------------------------------------------|
| <code>f.write(s)</code>          | 向文件写入一个字符串或字节流 <code>s</code>                         |
| <code>f.writelines(lines)</code> | 将字符串列表 <code>lines</code> 写入文件，不自动添加换行符               |
| <code>f.seek(offset)</code>      | 设置文件当前操作位置， <code>offset</code> 为偏移量，用于读写操作中改变当前操作的位置 |



# 举例：写文件

## • 举例：向文件中写入内容

1. `file = open('../data/b.txt', 'w+')`
  2. `list1 = ['Beautiful is better than ugly.', 'Explicit is better than implicit.', 'Simple is better than complex.']`
  3. `file.writelines(list1)`
  4. `for line in file:`
  5. `print(line)`
  6. `file.close()`
- 程序执行后，文件b.txt成功生成，并且列表的内容被写入了该文件。但是，程序在执行第4-5行时并没有输出任何内容。这是因为在第3行写入文件之后，文件指针移动到了文件的末尾，因此在第4-5行尝试读取时，因文件指针已经在文件末尾，它无法读取到任何内容
  - 要解决这个问题，可在原有的第4行之前添加一条语句`file.seek(0)`。这会将文件指针重新定位到文件的开始处，使得程序能够正确地打印出文件的内容



## 4. 二进制文件内容操作

- **二进制文件：**

- 二进制文件包括图像文件、音频文件、视频文件、数据库文件、可执行文件等。与文本文件不同，二进制文件包含的不仅仅是纯文本，而是可以包含任何类型的数据

- **字符串与字节序列的转换**

- 在处理文本文件和二进制文件时，经常需要在**字符串 (str)** 和**字节序列 (bytes)** 之间进行转换。**文本文件通常处理字符串数据，而二进制文件处理的是字节序列**
  - **字符串 → 字节序列**：在写入二进制文件之前，需要将字符串数据转换为字节序列。这通过字符串的 **.encode()** 方法完成，通常需要指定一个字符编码（如 UTF-8）
  - **字节序列 → 字符串**：在从二进制文件读取数据时，读取到的是字节序列。若需将这些数据转换为可读的字符串，可以使用字节序列的 **.decode()** 方法，同样需要指定字符编码



# 举例：二进制文件内容操作

- 示例:

- # 字符串转换为字节序列
- `str_data = "Hello, Python!"`
- `bytes_data = str_data.encode('utf-8')`
- # 将字节序列写入二进制文件
- `with open('example.bin', 'wb') as file:`
- `file.write(bytes_data)`
- # 从二进制文件中读取字节序列
- `with open('example.bin', 'rb') as file:`
- `bytes_data_read = file.read()`
- # 字节序列转换为字符串
- `decoded_str = bytes_data_read.decode('utf-8' )`
- `print(decoded_str)`
- 在这个示例中，首先将字符串数据编码为字节序列，然后写入一个二进制文件。读取二进制文件时，再将读取到的字节序列解码回字符串。这种转换在处理非文本数据（如图像、音频文件等）时尤为关键



## 5. 序列化与反序列化

- 在 Python 中，处理二进制文件常涉及序列化（编码）与反序列化（解码）的过程。
  - **序列化**：允许将 Python 对象转换为二进制格式，以便存储在文件中
  - **反序列化**：从文件中读取二进制数据并转换回 Python 对象
- **Python 提供了多种序列化模块**，如 pickle、json 等
  - **pickle 模块**：在 Python 中用于序列化和反序列化对象结构。这种机制允许将几乎所有的 Python 数据类型转换成字节流，并且可以从这些字节流中重构原始对象
    - pickle 在 Python 数据持久化方面非常有用，但生成的序列化数据不是为了人类阅读而设计的。虽然 pickle 非常强大，但它主要是 Python 专用的，因此不适合用于需要跨语言或跨平台的数据交换
  - **json 模块**：处理与 JSON（JavaScript Object Notation）数据格式相关的序列化和反序列化。它能够将 Python 数据结构转换成 JSON 格式字符串，这些字符串是文本格式并且可读性强。同样，它可以将 JSON 字符串解析回 Python 数据结构。
    - 由于其文本基础的性质和广泛的语言支持，json 是 Web 应用中数据交换的首选格式，非常适合用于各种应用程序之间的通信。





- 以 pickle 为例

- `import pickle`
- `flowerlist = ['rose', 'tulip', 'sunflower','morning glory']`
- `filename = 'flowlist.data'` #文件扩展名可自定义
- `file = open(filename, 'wb')` #以 “二进制写入” 模式打开文件
- `pickle.dump(flowlist, file)` #将内存中的列表内容写入外存中的文件
- `file.close()`
- `file = open(filename, 'rb')` #以 “二进制读取” 模式打开文件
- `storedflowerlist = pickle.load(file)` #将文件内容赋值给列表
- `print(storedflowerlist)`
- 输出: `['rose', 'tulip', 'sunflower','morning glory']`



## 6. 文件的操作

- 在Python中，除了文件内容的操作外，还可以对**文件本身进行操作**：
  - 设置文件权限、删除文件、重命名文件、检查文件属性和处理文件路径等。这些操作主要通过**os模块**和**os.path模块**实现
- **os模块常用文件操作**
  - os模块提供了许多与操作系统交互的功能，包括文件和目录的操作

| 函数                   | 说明                |
|----------------------|-------------------|
| os.chmod(path, mode) | 改变文件或目录的模式（权限）    |
| os.remove(path)      | 删除指定的文件           |
| os.rename(src, dst)  | 将文件或目录从src重命名为dst |
| os.stat(path)        | 获取文件或目录的状态信息      |
| os.listdir(path)     | 列出指定目录下的文件和目录列表   |



# 文件操作

- **os.path**模块常用文件操作

- os.path模块提供了一些操作和查询路径的函数。这些函数通常用于处理文件路径字符串，而不直接与文件系统交互

| 函数                              | 说明                  |
|---------------------------------|---------------------|
| os.path.exists(path)            | 检查指定路径的文件或目录是否存在    |
| os.path.isabs(path)             | 检查指定路径是否为绝对路径       |
| os.path.isdir(path)             | 检查指定路径是否为目录         |
| os.path.isfile(path)            | 检查指定路径是否为文件         |
| os.path.split(path)             | 将路径分割为目录和文件名，返回一个元组 |
| os.path.join(path1, path2, ...) | 将多个路径组合后返回          |



# 举例：文件操作

- 示例：使用os和os.path模块

- `import os`
- `# 检查文件是否存在`
- `if os.path.exists('example.txt'):`
- `print('文件存在')`
- `# 重命名文件`
- `if os.path.isfile('old_name.txt'):`
- `os.rename('old_name.txt', 'new_name.txt')`
- `# 列出当前目录下的所有文件和目录`
- `for item in os.listdir('.')`
- `print(item)`



## 7. 目录的操作

- 在Python中，os模块提供了丰富的目录操作功能，允许执行如**创建、删除目录，列出目录内容，以及遍历目录结构**等操作
- 常用目录操作函数：**

| 函数               | 说明                                                                         |
|------------------|----------------------------------------------------------------------------|
| os.mkdir(path)   | 创建一个名为path的新目录。如果目录已存在，会抛出异常                                               |
| os.rmdir(path)   | 删除一个名为path的目录。目录必须为空，否则会抛出异常                                               |
| os.listdir(path) | 返回一个列表，包含path目录下的所有文件和子目录名                                                 |
| os.getcwd()      | 返回当前工作目录的路径                                                                |
| os.walk(top)     | 生成一个目录树下的所有文件名。对于在目录top中的每个目录（包括它本身），产生一个3元组(dirpath, dirnames, filenames) |



## 示例：目录操作

- 列出当前目录下所有Python源文件

- `import os`
- `current_dir = os.getcwd()`
- `python_files = [f for f in os.listdir(current_dir) if os.path.isfile(f) and f.endswith('.py')]`
- `print(python_files)`
- 这个示例列出了当前工作目录下所有以`.py`结尾的文件

- 遍历目录树

- `import os`
- `for root, dirs, files in os.walk("d:/data"):`
- `for name in files:`
- `print(os.path.join(root, name))`
- `for name in dirs:`
- `print(os.path.join(root, name))`
- 这个示例遍历位于`d:/data`的目录树，打印出每个文件和目录的完整路径



## 8. 应用案例

- **常用的表格数据：**是以二维数据格式存储于文件。
  - 表格数据文件是有多行序列数据组成，每一行序列数据是由多个字段数据组成。常见的表格数据文件有CSV、Excel等。
  - CSV (Comma Sperated Values) 是一种国际通用的二维数据文件格式，一行数据中的每个字段之间用逗号隔开。
- **csv.reader()函数：**从一个打开的文件对象中读取数据，并将其转换为迭代器，该迭代器输出每一行的数据作为字符串列表
  - `import csv`
  - `with open('example.csv', 'r', newline='', encoding='utf-8') as file:`
  - `reader = csv.reader(file)`
  - `for row in reader:`
  - `print(row)`

`open()` 函数中的 `newline=""` 参数用于指定新行字符的处理方式。读取 CSV 文件时设置 `newline=""` 可以避免在 Windows 系统中多余的空行被添加，确保文件读取和写入的兼容性；`encoding='utf-8'` 则指定文件编码格式为 UTF-8，以避免读取包含非 ASCII 字符的文件时出现错误。



- **csv.writer() 函数**: 创建一个对象, 该对象能够将列表编码为 CSV 格式并写入到一个文件中。通常与 `writerow()` 和 `writerows()` 方法结合使用, 分别用于写入单行和多行数据
  - `import csv`
  - `with open('output.csv', 'w', newline='', encoding='utf-8') as file:`
  - `writer = csv.writer(file)`
  - `writer.writerow(['Header1', 'Header2'])`
  - `writer.writerows([[ 'data1', 'data2'], [ 'data3', 'data4']])`





# JSON数据文件处理

- **Python内置的json模块：**为处理JSON数据提供了强大的工具集
  - 它不仅支持将Python中的数据结构（例如字典和列表）转换为JSON格式字符串
  - 还可以将JSON字符串反序列化为Python对象
- **场景：**有一个包含多个用户信息的JSON文件。每个用户的数据包括个人信息、联系方式和兴趣爱好等多层次的数据。我们的目标是读取这个文件，提取并更新特定信息，最后将这些更改写回到一个新的JSON文件中。
  - 用户数据示例 (users.json)

```
[
  {
    "name": "John Doe",
    "age": 30,
    "contact": {
      "email": "johndoe@example.com",
      "phone": "123-456-7890"
    },
    "hobbies": ["reading", "hiking", "coding"]
  },
  {
    "name": "Anna Smith",
    "age": 25,
    "contact": {
      "email": "annasmith@example.com",
      "phone": "987-654-3210"
    },
    "hobbies": ["painting", "traveling", "dancing"]
  }
]
```



- **实现步骤:**

- **1. 读取JSON文件:** 利用Python的 json模块来读取users.json文件
- `import json`
- `with open('users.json', 'r') as file:`
- `users = json.load(file)`
- 使用json.load()函数, 将JSON数据读取为Python对象
- **2. 处理数据:** 对读入的数据进行处理, 比如增加每个用户的年龄, 或添加新的兴趣爱好。
- `for user in users:`
- `user["age"] += 1 # 增加每个用户的年龄`
- `if "coding" not in user["hobbies"]:`
- `user["hobbies"].append("coding") # 为兴趣爱好列表添加新项`
- **3. 写入更新后的数据到新文件:** 将修改后的数据写入新的JSON文件
- `with open('updated_users.json', 'w') as file:`
- `json.dump(users, file, indent=4)`
- 使用json.dump()函数将更新后的数据写入到updated\_users.json文件中, indent=4参数确保输出格式的可读性



# 本章习题

## 一、理论知识

1. 描述文本文件和二进制文件的主要区别。
2. 列举open()函数中几种常用的文件打开模式及其含义。
3. 解释为什么操作完文件后需要关闭文件，不关闭会有什么后果？
4. 讨论使用上下文管理器（with语句）操作文件的优势。
5. 解释绝对路径和相对路径的区别，并给出各自的使用场景。
6. 如何使用Python更改当前工作目录？
7. 使用os模块列举出如何在Python中创建新目录、删除文件和重命名文件。
8. 描述os.walk()函数的用途和工作方式。
9. 在处理文本文件时，为什么要关注文件的编码格式？
10. 解释字节序列和字符串在Python中的转换过程。



## 二、编程实践

1. 编写一个程序，读取一个文本文件，统计文件中每个单词的出现次数，并将结果写入另一个文件。
2. 编写程序遍历指定目录下的所有文件和子目录，并打印出它们的路径。
3. 已有一个 JSON 文件 `users.json`，里面存着一组用户的信息。现在要写一个 Python 程序做三件事：
  - (1) 读取 `users.json` 文件里的数据；
  - (2) 把第一个用户的城市改成 "Shanghai"（如果有用户的话）；
  - (3) 把修改后的数据写入到一个新的文件 `users_updated.json` 中。

【给定数据示例 (`users.json`) 】

```
[ {"name": "Zhang San", "city": "Beijing"},  
  {"name": "Li Si", "city": "Guangzhou"}  
]
```



4. 假设你正在设计一个小型的植物收藏游戏，玩家可以收集各种植物，将它们存储在文件中，并在下一次游戏启动时加载植物数据，继续他们的收藏。为了实现这一功能，需要设计一个简单的存储和加载系统，以便玩家的植物数据能够持久化。

要求：

- 1) 使用 pickle 模块实现植物收藏的序列化和反序列化功能。
- 2) 每次游戏启动时加载存储的植物数据，如果文件不存在则创建一个空的收藏。
- 3) 提供以下功能：
  - 添加植物：玩家可以输入植物名称，将其添加到收藏中。
  - 显示收藏：展示玩家当前收藏的所有植物。
  - 保存收藏：将当前的收藏数据序列化并存储到文件中。



**The End !**